TapMind Integration Documentation — Page Content

Purpose: the detailed copy to be added to each page of the JAMdesk integration doc. **Format:** paste into Google Docs (enable *Tools → Preferences → Automatically detect Markdown*, or *File → Import*).

How to read this document

The docs are built **DRY**: anything identical across pages is written **once** here and reused. So this document is organized by *building block*, not by repeating all 33 pages. Each mediation page is assembled from a fixed recipe (below), so you have full content for every page without duplication.

⚠ Values still blocked on dev verification

The following are written as **placeholders** and must be filled from the *Class & Network Key Registry* once the dev verification checklist returns. Do **not** invent them:

- `«CLASS:{mediation}-{platform}»` — adapter class string (Checklist Q2–Q8)
- `«NETWORK-KEY»` — LevelPlay network key (declared canonical `15c11cb1d`; packages must be updated to match)
- `«VERSION:{package}»` — pinned package version (Compatibility Matrix)
- `«MIN-IOS» / «MIN-ANDROID»` — confirmed OS floors (Q17/Q18)

Page assembly recipe (every Custom Adapter mediation page)

Page = Step 1 + Step 2 + Step 3 + Step 4 + Step 5 + Step 6, in this order:

1. **Prerequisites** → *Prerequisites snippet* (per framework)
2. **Install the SDK** → *per-cell install block* (from the install table)
3. **Configure your app** → *App-ID snippet* (Android or iOS)
4. **Configure {mediation}** → *Dashboard-config snippet* (per mediation)
5. **Verify** → *Verify snippet*
6. **Troubleshoot** → *Troubleshoot snippet*

PART 1 — Front-door pages (full content)

Page: Introduction (`index`)

Title: Introduction **Description:** Integrate TapMind as your ad mediation demand partner across native mobile, game engines, and cross-platform frameworks.

Body:

TapMind connects your app to premium ad demand through your existing mediation stack. You add TapMind as a custom adapter inside the mediation platform you already use — AdMob, Google Ad Manager, AppLovin MAX, or Unity LevelPlay — and TapMind competes for your ad requests like any other network.

Choose your SDK

- **Custom Adapter GMA SDK** — a lightweight adapter for an existing mediation stack. Android, iOS, and four cross-platform frameworks.
- **Custom Adapter Next-Gen SDK** — native Android integration for AdMob and Google Ad Manager on the next-generation GMA API.
- **Orchestration SDK** — lightweight Android ad serving through placement codes, with no third-party mediation platform.

Supported platforms: Native Android, Native iOS, Flutter, React Native, Unity, Cocos.

How to use these docs: Pick your platform, then your SDK product, then the mediation page for the partner you use. Every page follows the same six steps: prerequisites → install → configure your app → configure the mediation dashboard → verify → troubleshoot.

Need help? Contact your TapMind account manager for the placement values and configuration specific to your account.

Page: How custom adapters work (`concepts`) — NEW

Title: How TapMind custom adapters work **Description:** The mental model that makes every integration page make sense.

Body:

The one idea to hold onto: with the Custom Adapter SDKs, you do **not** call TapMind directly. You integrate your mediation SDK exactly as you normally would, and TapMind rides inside it as a *custom network*. Your mediation platform loads the TapMind adapter and asks it for an ad whenever TapMind's line item is eligible.

The request flow:

```
None
Your app  →  Mediation SDK (AdMob / MAX / LevelPlay / GAM)
          →  picks TapMind from the waterfall
          →  loads the TapMind adapter class
          →  TapMind backend returns an ad
          →  ad renders in your app
```

What this means in practice:

- You add TapMind in **two places**: a dependency in your build, and a custom-event/custom-network entry in your mediation dashboard.
- After that, you request ads through your **mediation SDK's normal API** — there is no separate TapMind "load ad" call (the Orchestration SDK is the exception; it has its own API).
- TapMind only serves when your waterfall reaches its line item, at the eCPM you configure.

Key terms:

Term	Meaning
Custom event / custom network	The mechanism your mediation platform uses to call a third-party adapter like TapMind.
Class name	The exact adapter class your mediation dashboard loads. Must match the published SDK exactly.
`placementName`	A per-placement identifier you enter in the dashboard. Provided by your account manager. Case-sensitive, exact-match.
Network key	A per-account key that identifies TapMind in Unity LevelPlay.

Term	Meaning
eCPM / rate	The manual price that sets TapMind's position in your waterfall.

> **Note:** This page is the prerequisite mental model for every integration page. If an ad isn't serving, 90% of the time it traces back to one of the four bolded values above not matching exactly.

Page: Choose your SDK (`choose-your-sdk`) — NEW

Title: Choose your SDK **Description:** Pick the right TapMind product for your monetization setup.

Body:

If you…	Use	Platforms
Already run a mediation stack (AdMob, MAX, LevelPlay, or GAM) and want TapMind as additional demand	**Custom Adapter GMA SDK**	Android, iOS, Flutter, React Native, Unity, Cocos
Run AdMob or GAM on Android and want the next-generation GMA integration	**Custom Adapter Next-Gen SDK**	Android only
Want TapMind to serve ads directly via placement codes, without a third-party mediation platform	**Orchestration SDK**	Android only

How to decide:

- **Most publishers use Custom Adapter GMA.** If you have a mediation account and an ad unit, this is your path.
- **Next-Gen** is for teams adopting Google's next-generation GMA API on Android; it covers AdMob and GAM only.

- **Orchestration** is a different model — TapMind serves directly, you call a TapMind API with a placement code, and there is no mediation dashboard step. Pick this only if you are not mediating through a third party.

Once you've chosen, open the matching section in the sidebar and follow the six-step flow for your framework and mediation partner.

Page: Prerequisites (`prerequisites`) — NEW

Title: Prerequisites **Description:** What you need before you start any integration.

Body:

Before integrating any TapMind SDK, make sure you have:

- **A mediation account** with the partner you're integrating (AdMob, GAM, AppLovin MAX, or Unity LevelPlay).
- **An ad unit already created** in that platform — TapMind targets your existing ad units.
- **The Google Mobile Ads (GMA) SDK** integrated in your app (TapMind's adapters run on top of it).
- **Your TapMind values**, provided by your account manager (see the split below).

Two kinds of values — know which is which:

You set these	**Your account manager provides these**
Your app ID, your ad unit	`placementName`
Your eCPM/waterfall position	Adapter class name (or LevelPlay network key)
	Recommended eCPM / rate

All TapMind-provided values live in one place — the **Class & Network Key Registry** (Reference tab). Use it as your single source of truth.

Minimum OS: see the **Compatibility Matrix** for the confirmed minimum Android and iOS versions per package. *(Values pending dev confirmation — `«MIN-ANDROID»` / `«MIN-IOS»`.)*

Page: Gradle setup for Gradle 7+ (`gradle-setup/for-gradle-version-7-plus`) — FILL STUB

Title: Gradle setup (Gradle 7+) **Description:** Repository configuration for Android-based integrations on Gradle 7 and above.

Body:

On Gradle 7+, repositories are declared in `settings.gradle` under `dependencyResolutionManagement`, not in the project `build.gradle`:

```
None
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.PREFER_SETTINGS)
    repositories {
        google()
        mavenCentral()
        maven { url 'https://jitpack.io' }
    }
}
```

> **Note:** If your project uses this `settings.gradle` style, do **not** also add the `allprojects { repositories { … } }` block shown on individual mediation pages — choose one. *(Confirm the exact repositories TapMind requires with the dev team — Checklist Q24/Q25.)*

PART 2 — Mediation page building blocks (Custom Adapter GMA)

These blocks assemble into every Custom Adapter GMA page per the recipe above.

2.1 — Prerequisites snippet (per framework)

Standard prerequisites callout. Min-OS values are **pending confirmation**.

Framework	minSdk (Android)	min iOS	Other
Native Android	`«MIN-ANDROID»` (doc shows 23)	—	An ad unit must exist
Native iOS	—	`«MIN-IOS»` (12.0 per dev — confirm GMA floor)	An ad unit must exist
Flutter	`«MIN-ANDROID»` (24)	`«MIN-IOS»` (15.0)	GMA SDK integrated
React Native	`«MIN-ANDROID»` (24)	`«MIN-IOS»` (15.0)	GMA SDK integrated
Unity	per GMA plugin	per GMA plugin	GMA Unity plugin imported
Cocos	`«MIN-ANDROID»` (26)	`«MIN-IOS»` (repo states 18.0 — confirm)	—

> All four prerequisites callouts must state: an ad unit must already exist; the GMA SDK must be integrated.

2.2 — Android Gradle repositories snippet (shared, all Android-based pages)

```
None
allprojects {
    repositories {
        google()
        mavenCentral()
        maven { url 'https://jitpack.io' }
    }
}
```

(Or the Gradle 7+ `dependencyResolutionManagement` form — see the Gradle setup page. Use one, not both.)

2.3 — App-ID snippet — Android

Add your AdMob app ID to `AndroidManifest.xml`. **Omitting this crashes the app on launch** with `Missing application ID`.

```xml
<manifest>
  <application>
    <!-- Sample AdMob app ID: ca-app-pub-3940256099942544~3347511713 -->
    <meta-data
        android:name="com.google.android.gms.ads.APPLICATION_ID"
        android:value="YOUR_ADMOB_APP_ID"/>
  </application>
</manifest>
```

2.4 — App-ID snippet — iOS

Add `GADApplicationIdentifier` to `Info.plist`, plus the `SKAdNetworkItems` array for attribution.

```xml
<key>GADApplicationIdentifier</key>
<string>YOUR_ADMOB_APP_ID</string>
```

> Include the full `SKAdNetworkItems` list (Google + third-party buyer IDs) as currently published. Keep it in **one** shared snippet so it never drifts between pages.

2.5 — Verify snippet (shared, closes the loop)

After install and dashboard configuration:

1. Run your app in a **debug build**. TapMind's test mode is enabled automatically in debug builds and serves test ads at a high fill rate.
2. Request an ad through your mediation SDK as you normally would.
3. Confirm a TapMind ad fills. Check logs for the tag **`«LOG-TAG»`** *(confirm canonical tag with dev — currently `TapMindAdapter` on some pages)*.
4. When verified, build in **release** mode for production — test mode is disabled automatically in release builds, and live ads serve.

> **Test mode is automatic.** It follows your build type: debug = test ads, release = live ads. There is no flag to enable or disable, and nothing to turn off before going live.

2.6 — Troubleshoot snippet (shared)

Symptom	Likely cause	Fix
No TapMind ad in production, but works in debug	Wrong/missing class string, or release-build stripping	Verify the class string matches the Registry exactly; confirm ProGuard keep-rules ship *(dev — Checklist Q9)*
No fill at all	`placementName` mismatch (case/space) or wrong eCPM position	Re-check `placementName` exactly; confirm waterfall position
App crashes on launch	Missing AdMob app ID in manifest/Info.plist	Add the app-ID meta-data exactly as shown
LevelPlay network never appears	Wrong network key	Use the canonical key from the Registry

For anything else, contact your account manager with the log output and the tag `«LOG-TAG»`.

2.7 — Per-mediation dashboard config (written ONCE each)

AdMob dashboard config

Create the custom event (waterfall): Mediation → Create mediation group → select platform + ad format → name it (provided by your account manager) → Add ad units → select your app and ad units.

Add the ad source: Waterfall source → Set up ad source → Custom event → select your app/platform → select the ad unit, then enter:

```
None
Mapping name : provided by your account manager
eCPM         : provided by your account manager
Class        : «CLASS:admob-{platform}»
Parameter    : { "placementName": "provided by your account manager" }
```

> ⚠ `placementName` must match **exactly** — case-sensitive, no leading/trailing spaces. A mismatch fails silently (no error, no fill).

Google reference: https://support.google.com/admob/answer/13407144

AppLovin MAX dashboard config

Create the custom network: MAX → Mediation → Manage → Networks → "Add a Custom Network" → Network type **SDK** →

```
None
Custom Network : TapMind Market Place
Class name     : «CLASS:applovin-{platform}»
```

Add the yield partner: MAX → Mediation → Manage → Ad Units → select your ad unit → enable **TapMind** → enter:

```
None
App ID            : leave blank (optional)
Placement ID      : provided by your account manager
eCPM              : provided by your account manager
Custom Parameters : {"placementName":"provided by your account manager"}
```

> ⚠ `placementName` exact-match warning (as above).

Unity LevelPlay dashboard config

Create the TapMind network: Monetize → Setup → SDK Networks → Available Networks → Manage Networks → select **Custom Adapter** → enter network key **`«NETWORK-KEY»`** *(canonical `15c11cb1d`)* → Enter → Save. The network appears as **TapMind**.

Set up instances: Setup → Instances → select TapMind (Custom) → Add ad instance → select ad type →

```
None
Instance Name : provided by your account manager
eCPM / Rate   : provided by your account manager
```

> Enter the rate for each placement — it determines waterfall order. Add one instance per ad format to start.

Google Ad Manager (GAM) dashboard config

Create the TapMind company: Admin → Companies → New company → type **Ad Network** → name **TapMind** → Ad Network **Other Company** → toggle **Mediation** on → Save.

Create/open a yield group: Delivery → Yield groups → New yield group → name (from your account manager) → set ad format → inventory type **Mobile** → target your ad units.

Add the yield partner: Add yield partner → select **TapMind** → Integration type **Custom Event** → Status **Active** → set platform →

```
None
Default CPM : provided by your account manager
Label       : provided by your account manager
Class Name  : «CLASS:gam-{platform}»
Parameter   : { "placementName": "provided by your account manager" }
```

> ⚠ `placementName` exact-match warning (as above). **Note (current state):** native Android/iOS use the dedicated GAM class; the wrapper SDKs (Flutter, RN, Unity, Cocos) currently use the AdMob class for GAM. The Registry shows the correct string per platform.

Google reference: https://support.google.com/admanager/answer/7390828

2.8 — Per-cell install table (the only per-page-unique content)

For each page, Step 2 uses the row below. **All versions pending** (`«VERSION:…»`).

Native Android (`build.gradle`, module: app)

Mediation	Dependency
AdMob	`implementation("io.github.tapmind-tech:customadapter-admob:«VERSION»")`
AppLovin	`implementation("io.github.tapmind-tech:customadapter-applovin:«VERSION»")`

Mediation	Dependency
GAM	`implementation("io.github.tapmind-tech:customadapter-gam:«VERSION»")`
LevelPlay	`implementation("io.github.tapmind-tech:customadapter-ironsource:«VERSION»")`

Plus GMA:
`implementation("com.google.android.gms:play-services-ads:«VERSION»")`

Native iOS (Podfile / SPM)

Mediation	CocoaPods	SPM repo
AdMob	`pod 'TapMindAdapter'`	`TapMind-Custom-Adapter-iOS.git`
AppLovin	`pod 'TapMindALAdapter'`	`TapMind-CA-Applovin-iOS.git`
GAM	`pod 'TapMindAdapter'`	`TapMind-Custom-Adapter-iOS.git`
LevelPlay	`pod 'TapMindISAdapter'`	`«SPM repo – confirm; doc shows a Unity repo»`

SPM: add the `-ObjC` flag (Targets → Build Settings → Other Linker Flags).

Flutter (`pubspec.yaml`)

Mediation	Dependency
AdMob	`tap_mind_ads_admob: «VERSION»`
AppLovin	`tapmind_ads_applovin: «VERSION»`
GAM	`tap_mind_ads_admob: «VERSION»`
LevelPlay	`tapmind_ads_ironsource: «VERSION»`

Plus `google_mobile_ads: «VERSION»`. *(Confirm exact pub.dev names — Q16.)*

React Native (`package.json` → install via npm/yarn)

Mediation	Package
AdMob	`tapmind_ads_admob`
AppLovin	`tapmind_ads_applovin`
GAM	`tapmind_ads_gam`
LevelPlay	`tapmind_ads_ironsource`

Plus `react-native-google-mobile-ads`. **Fix:** use the real RN Podfile (your app's target, `use_native_modules!`) — not the Flutter `Runner` block. *(Confirm correct RN Podfile — Q20.)*

Unity (Package Manager → Git URL)

Mediation	Git URL
AdMob	`TapMind-CA-Admob-Unity.git«#TAG»`
AppLovin	`TapMind-CA-Applovin-Unity.git«#TAG»`
GAM	`TapMind-CA-Admob-Unity.git«#TAG»`
LevelPlay	`TapMind-CA-Ironsource-Unity.git«#TAG»`

> Pin a release tag (`«#TAG»`) — do not pull `main`. *(Tags pending — Q19.)* Also import the GMA Unity plugin and set AdMob app IDs in Assets → Google Mobile Ads → Settings.

Cocos (extension install)

Install the TapMind extension: Extension → Extension Manager → Install from File → select the TapMind extension ZIP → Enable. Pin a tagged release ZIP rather than `archive/refs/heads/main.zip`. *(Tags pending — Q19.)*

PART 3 — Custom Adapter Next-Gen (Android only)

Same 6-step recipe. Differences from CA-GMA:

- **Mediations:** AdMob and GAM only.
- **Install:**
  - AdMob: `implementation("io.github.tapmind-tech:customadapter-admob-nextgen:«VERSION»")`
  - GAM: `implementation("io.github.tapmind-tech:customadapter-gam-nextgen:«VERSION»")`
- **Class strings:** the Next-Gen namespace differs from CA-GMA — `«CLASS:nextgen-admob»` / `«CLASS:nextgen-gam»` (doc shows `com.tapmind.mediation.ng.*`). Confirm via Registry / Q7.
- **Dashboard config:** identical to the CA-GMA AdMob/GAM flows (Part 2.7), using the Next-Gen class strings.
- **Log tag:** doc shows responses under `TapMindAdapter` — confirm.

PART 4 — Orchestration SDK (Android only) — full content

Page: Overview

Title: Overview **Body:** The Orchestration SDK is a lightweight Android library that serves ads through defined placement codes. It lets you monetize specific moments in your app by associating placements with screen transitions or user interactions. Unlike the Custom Adapters, there is no third-party mediation dashboard — TapMind serves directly, and you call the SDK with a placement code.

Page: Requirements & Setup

Prerequisites: Android API level `«MIN-ANDROID»` (24 / Android 7.0) or higher; Google Play Services available on device.

Add the dependency (app-level `build.gradle`):

```
None
implementation("io.github.tapmind-tech:orchestration:«VERSION»")
```

Core dependencies (used internally): Gson (serialization), Lifecycle Process (foreground/background), Play Services Ads (rendering).

Manifest: add your Google Ad Manager app ID:

```
XML
<meta-data
    android:name="com.google.android.gms.ads.APPLICATION_ID"
    android:value="YOUR_GAM_APP_ID"/>
```

Page: Initialize the SDK

Initialize once at app startup, inside your custom `Application` class:

```
Kotlin
class MyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        val orchConfig = OrchConfig.Builder()
            .testMode(BuildConfig.DEBUG)
            .build()

        OrchSdk.initialize(
            this,
            orchConfig,
            object : OrchInitListener {
                override fun onInitSuccess() { /* ready */ }
                override fun onInitError(error: OrchError) {
                    Log.e("ORCH", "Init failed: ${error.message}")
                }
            }
        )
    }
}
```

> Register `MyApp` in `AndroidManifest.xml` via `android:name`.

Parameters: `context` (use application context to avoid leaks), `config` (`OrchConfig`), `listener` (`OrchInitListener`). **`OrchConfig` methods:** `testMode(Boolean)` — debug builds serve test ads, release builds serve live ads, automatically.

Page: Load & Show Ads

```kotlin
OrchAd.setListener(this)
OrchAd.loadAd(this, "<placement_code>")
```

Placement codes are provided and managed by TapMind. Initialize before calling `loadAd()`.

Lifecycle callbacks (`OrchAdListener`):

Callback	Fires when
`onAdLoaded()`	Ad is ready to display
`onAdImpression()`	Impression recorded
`onAdClicked()`	User clicked
`onAdDismissed()`	Ad closed
`onAdFailedToLoad(error)`	Load failed (see Error Reference)
`onAdFailedToShow(error)`	Loaded but failed to display

Page: Privacy & Consent

The Orchestration SDK is compatible with Google's User Messaging Platform (UMP) for consent.

```
implementation("com.google.android.ump:user-messaging-platform:«VERSION»")
```

Before the first ad request: create a `ConsentInformation` instance → `requestConsentInfoUpdate(...)` → `loadAndShowConsentFormIfRequired(...)`. The form shows only when required by the user's region; otherwise UMP completes the flow automatically.

Page: Error Reference

Code	Name	Meaning	Action
1001	`SDK_NOT_INITIALIZED`	Ad requested before init completed	Wait for `onInitSuccess()` before `loadAd()`
1002	`INVALID_PLACEMENT`	Empty/invalid placement code	Match the code provided by TapMind
1003	`NO_NETWORK`	No connectivity	Check connection, retry
1004	`CONFIG_ERROR`	Config request failed/timed out	Retry; contact support if persistent
1005	`NO_FILL`	No ad available	Normal; retry later or use fallback
1006	`AD_EXPIRED`	Ad expired	Handled automatically
1007	`PKG_MISSING`	Package name missing/invalid	Verify init params and config
1008	`PLAY_SERVICE_MISSING`	Play Services unavailable	Ensure Play Services installed/updated
1009	`UNSUPPORTED_VERSION`	Android version unsupported	Use a supported Android version

Best practices: initialize before requesting; verify placement codes; handle `onAdFailedToLoad`/`onAdFailedToShow`; log code + message together when contacting support.

Page: Support

Contact your TapMind account manager for Orchestration SDK support. Include the error code and message.

PART 5 — Reference pages

Page: Class & Network Key Registry — NEW (single source of truth)

Title: Class & network key registry **Description:** The canonical class strings and network key for every SDK, platform, and mediation. All other pages reference this.

> ⚠ Fill from the dev verification checklist (shipped artifacts). Until confirmed, treat every cell as pending.

SDK	Mediation	Android class	iOS class
CA-GMA	AdMob	`«CLASS:admob-android»`	`«CLASS:admob-ios»`
CA-GMA	AppLovin	`«CLASS:applovin-android»`	`«CLASS:applovin-ios»`
CA-GMA	GAM (native)	`«CLASS:gam-android»`	`«CLASS:gam-ios»`
CA-GMA	GAM (wrappers)	`«CLASS:gam-wrapper-android»`	`«CLASS:gam-wrapper-ios»`
Next-Gen	AdMob	`«CLASS:nextgen-admob»`	n/a
Next-Gen	GAM	`«CLASS:nextgen-gam»`	n/a

LevelPlay network key (all platforms): `«NETWORK-KEY»` — canonical `15c11cb1d`.

Page: Compatibility Matrix — NEW

Title: Compatibility matrix Columns: TapMind package + version · required GMA SDK version · required mediation SDK version · min Android · min iOS. One row per published package. Fill from Checklist Q15/Q17/Q18/Q23.

Page: Troubleshooting — NEW

Use the shared troubleshoot table (2.6) plus:

- **Release vs debug:** if it works in debug but not release, suspect the class string or release-build stripping. *(ProGuard keep-rule confirmation — Q9.)*
- **Diagnostic logging:** filter logcat by `«LOG-TAG»`.
- **Escalation:** include the tag output, the mediation platform, the class string used, and the `placementName`.

Page: Privacy & Consent (all SDKs) — NEW

Title: Privacy & consent Document how consent signals (GDPR/TCF, US privacy, iOS ATT/IDFA) reach TapMind through the adapters, and the iOS Privacy Manifest / Play Data Safety declarations. *(Mechanism pending product confirmation — currently only UMP-for-Orchestration is documented.)*

Appendix — Placeholder legend

Placeholder	Resolves to	Source
`«CLASS:…»`	Adapter class string	Registry / Checklist Q2–Q8
`«NETWORK-KEY»`	LevelPlay key (`15c11cb1d`)	Locked; packages to match
`«VERSION:…»`	Pinned package version	Compatibility Matrix / Q15
`«MIN-IOS» / «MIN-ANDROID»`	Confirmed OS floor	Q17 / Q18
`«LOG-TAG»`	Canonical log tag	Dev confirm
`«#TAG»`	Unity/Cocos release tag	Q19

Branding: use "TapMind" (not "TapMinds") and "Unity LevelPlay" (not "IronSource") throughout, coordinated with the rename sprint. **Jargon:** replace "G-sheet" with "your account manager" everywhere.